

Object Oriented Concepts || P3

Rahul Guha || SME || Java FS Training || HCL

Inheritance

Inheritance in Java is a core OOP concept that allows a class to acquire properties and behaviors from another class. It helps in creating a new class from an existing class, promoting code reusability and better organization.

- A subclass can reuse the fields and methods of the parent class without rewriting the code
- A subclass can add its own fields and methods or modify existing ones to extend functionality.

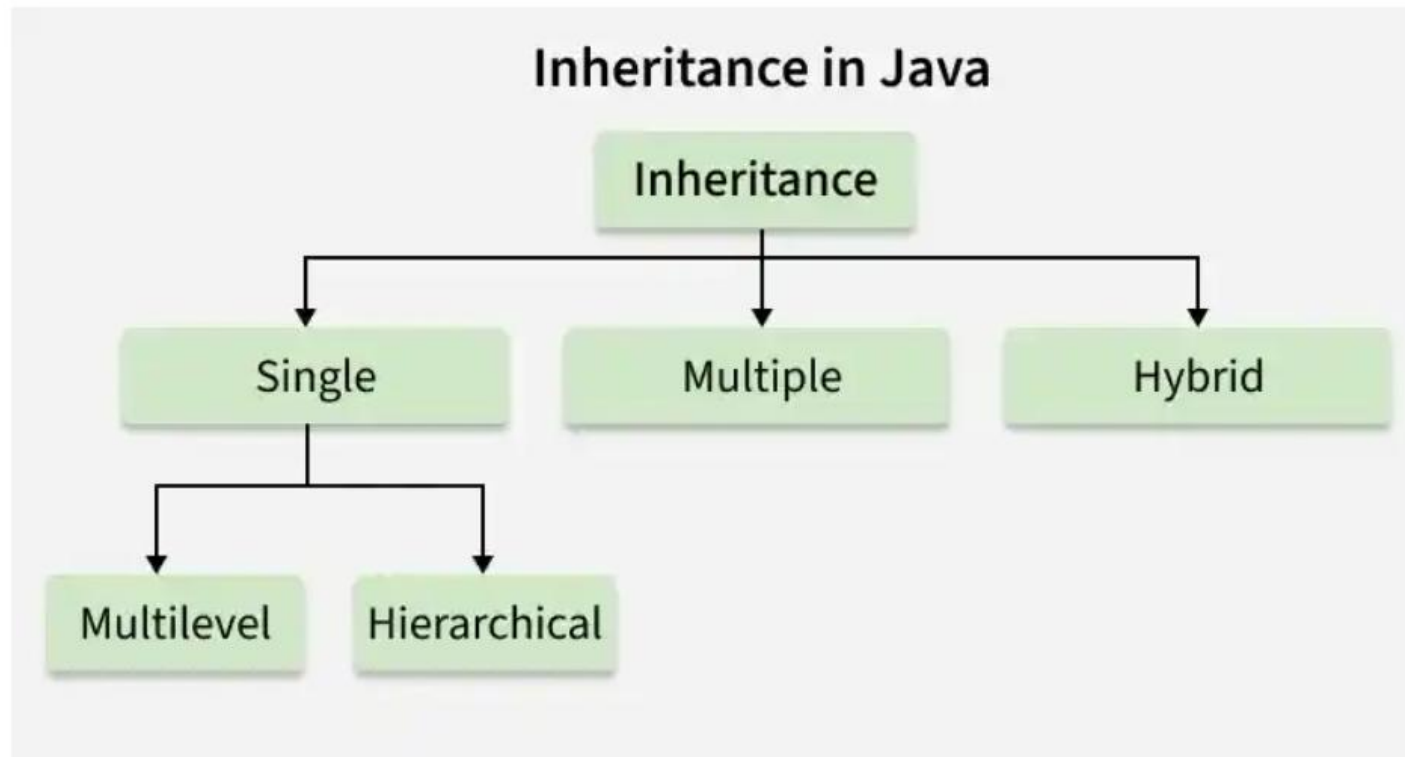
Key Terminologies in Java Inheritance

- **Class** : Blueprint from which objects are created
- **Superclass** : Class whose properties are inherited
- **Subclass** : Class that inherits another class
- **extends** : Keyword used to inherit a class

Inheritance

Types of Inheritance in Java

It defines the different ways a class can inherit properties and behavior from one or more classes.



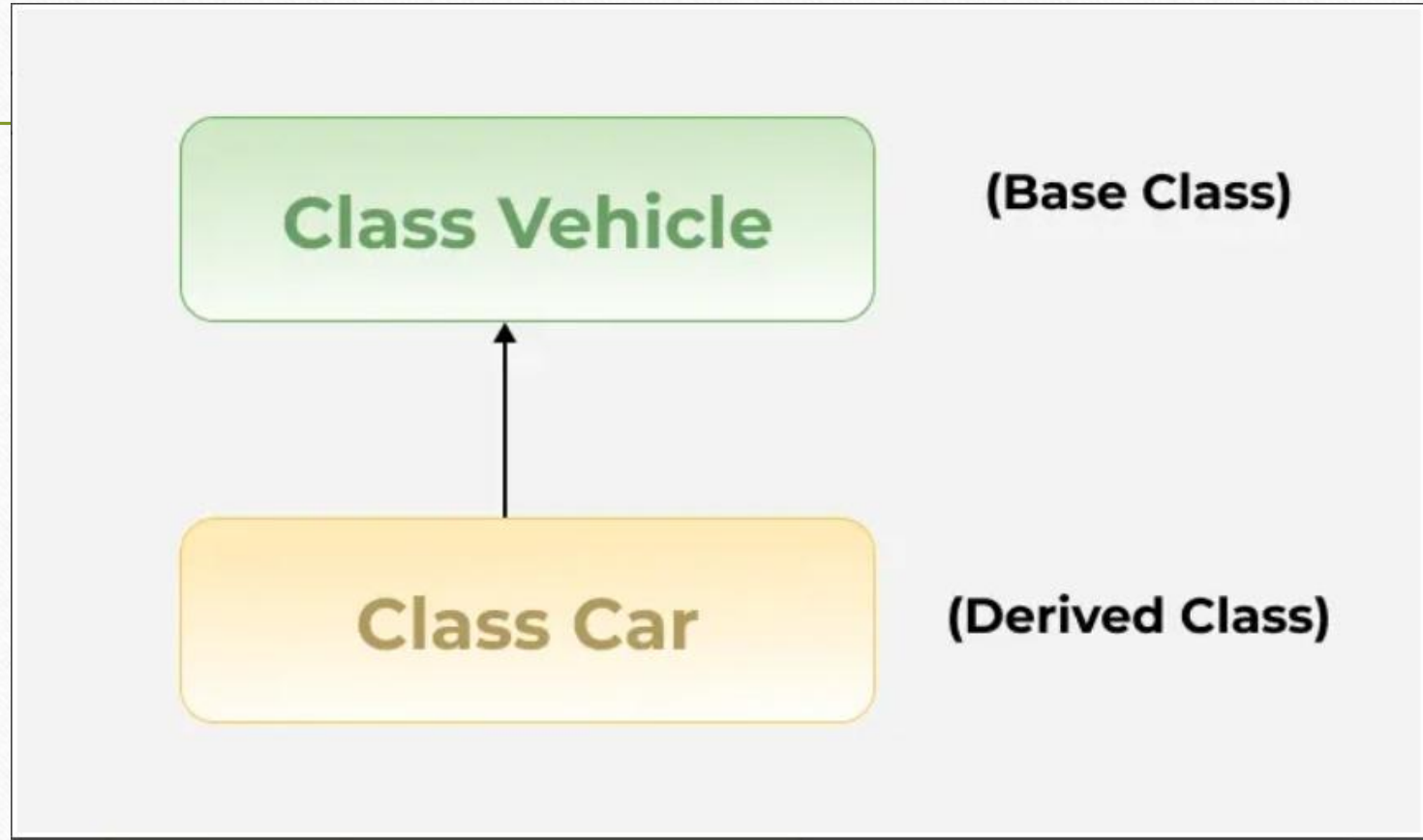
Types of Inheritance in Java

Introduction

- Inheritance means acquiring the properties and behaviors of a parent class in a child class.
- It allows a subclass (child class) to inherit fields and methods from a superclass (parent class), promoting code reuse and method overriding.

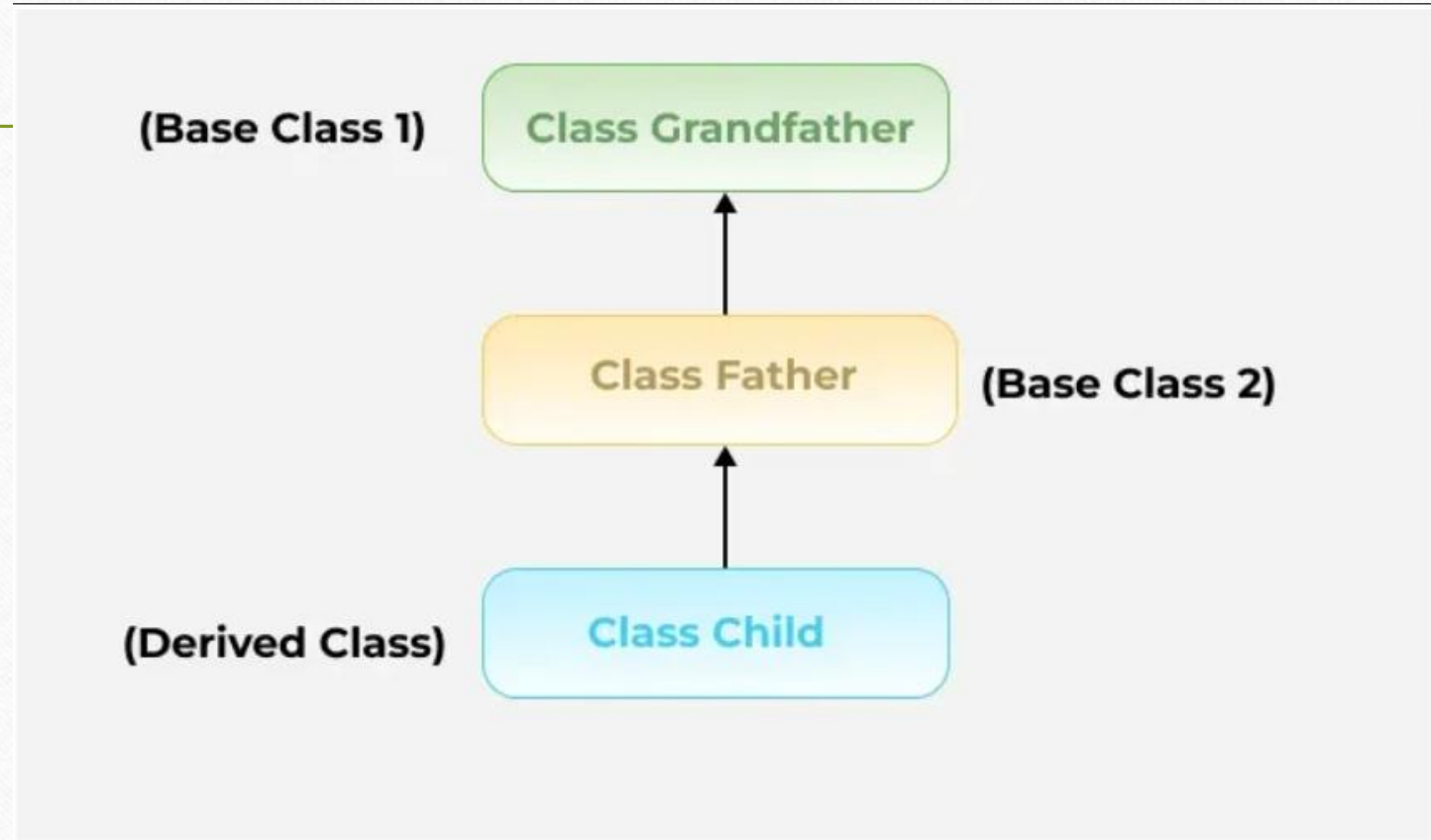
1. Single Inheritance

In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behavior of a single-parent class. Sometimes, it is also known as simple inheritance.



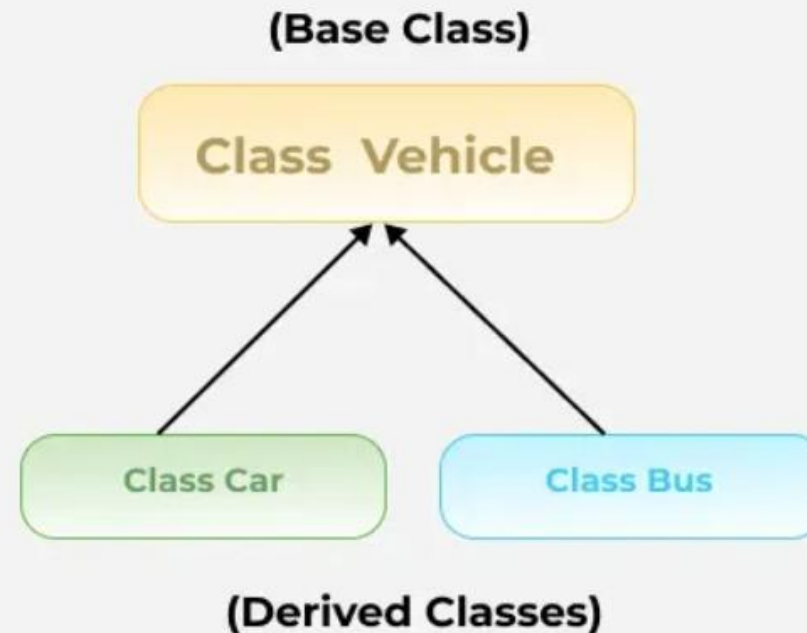
2. Multilevel Inheritance

In Multilevel Inheritance, a derived class will be inheriting a base class and as well as the derived class also acts as the base class for other classes.



3. Hierarchical Inheritance

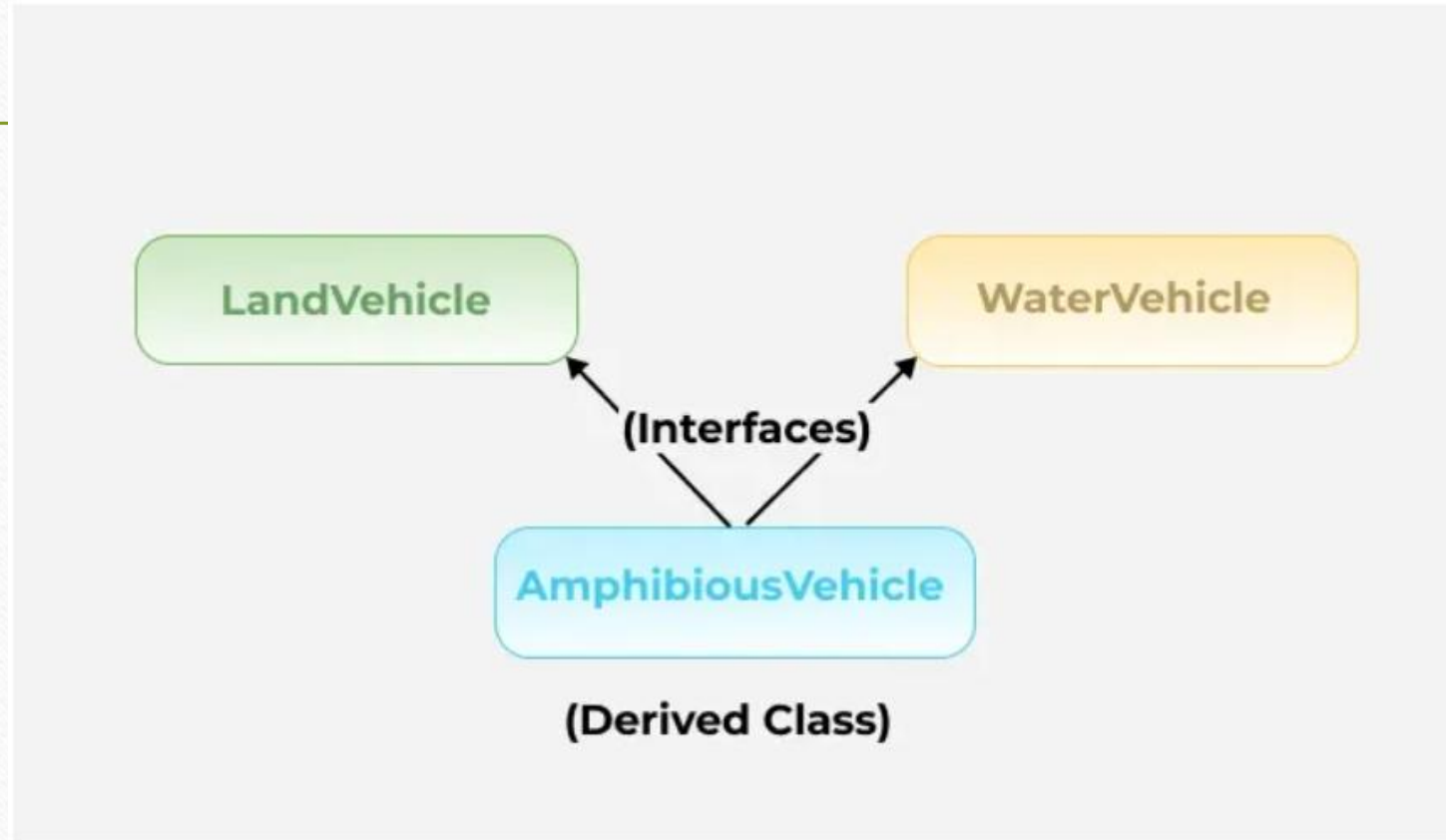
In hierarchical inheritance, more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class. For example, cars and buses both are vehicle.



4. Multiple Inheritance (Through Interfaces)

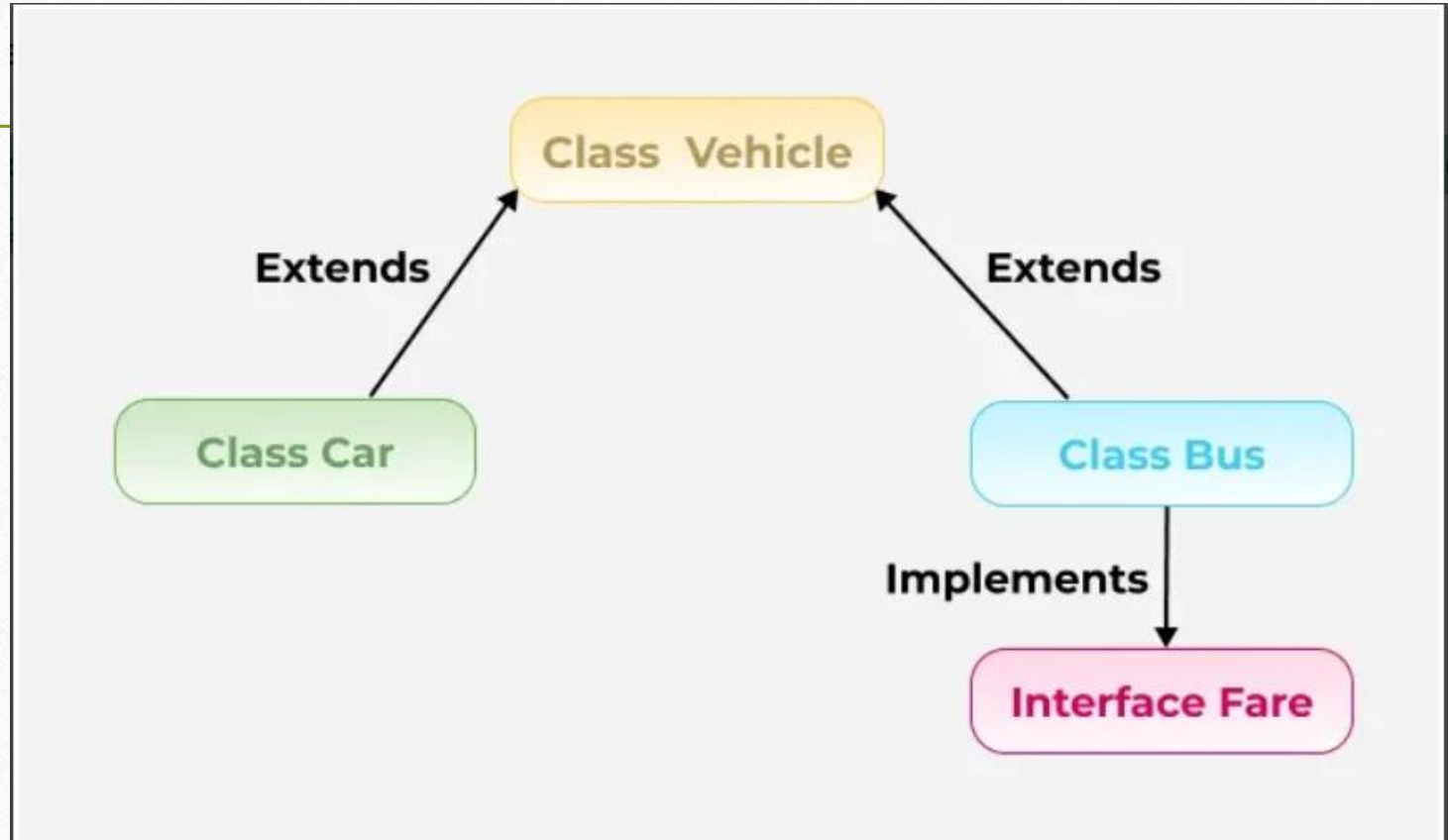
In Multiple inheritances, one class can have more than one superclass and inherit features from all parent classes.

Note: that Java does not support multiple inheritances with classes. In Java, we can achieve multiple inheritances only through Interfaces.



5. Hybrid Inheritance

It is a mix of two or more of the above types of inheritance. In Java, we can achieve hybrid inheritance only through Interfaces if we want to involve multiple inheritance to implement Hybrid inheritance.



What is an interface?

- An **interface** in Java is a blueprint that defines behavior that a class can agree to provide.

```
interface fare {
```

- - default void fareInfo() {
 - System.out.println("The Fare in Bus is nominal");
 - }
 - }
- means that fare is an **interface**, not a normal class.

- **Why did we create fare as an interface?**
Our Vehicle class represents general vehicle functionality:

```
class Vehicle {
```

- void engine() {
 - System.out.println("It's a Masterclass Engine");
 - }
 - }
- Maruti & Bus are vehicles, so both can inherit: engine()

What does implements mean?

This is the most important part:
class bus extends Vehicle implements fare
There are actually **two relationships** here.

First: extends Vehicle :
means -> bus inherits from the Vehicle class.
Therefore bus gets: engine()

Second: implements fare:
means:
bus implements the fare interface. fareInfo()
Therefore bus can use:fareInfo()

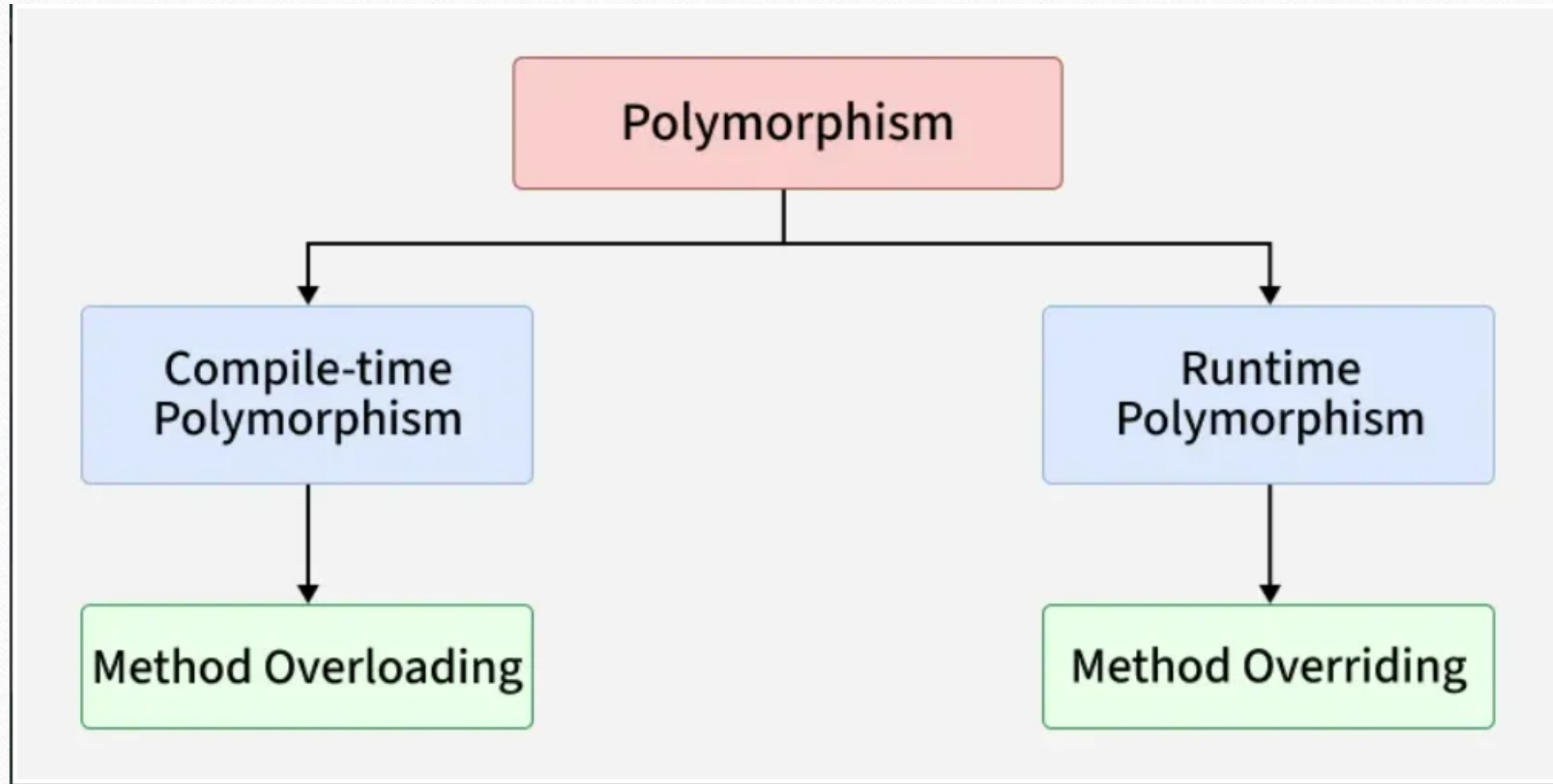
extends	implements
Used for class inheritance	Used for interface implementation
Class → Class	Class → Interface
Inherits from a class	Implements an interface
A class can extend one class	A class can implement multiple interfaces
Example: Bus extends Vehicle	Example: Bus implements Fare

Java Polymorphism

Polymorphism means "many forms", and it occurs when we have many classes that are related to each other by inheritance.

- Like we specified in the previous chapter; Inheritance lets us inherit attributes and methods from another class. Polymorphism uses those methods to perform different tasks. This allows us to perform a single action in different ways.
- Polymorphism means "many forms".
- It allows the same method to exhibit different behaviors.
- It is achieved through method overloading and method overriding.

Types of Polymorphism in Java



1. Compile-Time Polymorphism

- **Compile-time polymorphism** also known as static polymorphism, occurs when the method to be executed is determined during compilation. It is primarily achieved through method overloading.
- Method selection occurs at compile time.
- Improves readability and flexibility.

Method Overloading

- Method overloading allows multiple methods with the same name but different parameter lists within the same class. The compiler chooses the appropriate method based on the arguments passed. The parameters can differ in:
- Number of parameters
- Data types of parameters
- Order of parameters

2. Runtime Polymorphism

Runtime polymorphism also known as dynamic polymorphism, occurs when the method call is resolved during program execution. It is achieved through method overriding.

- Method selection occurs at runtime.
- Depends on the actual object type.

Method Overriding

- Method overriding occurs when a subclass provides its own implementation of a method already defined in its superclass. The overridden method is selected based on the actual object created.

Type	Also called	Achieved by
Compile-time	Static polymorphism	Method Overloading
Runtime	Dynamic polymorphism	Method Overriding

So what is the purpose of `@Override`?

`@Override` is an annotation that tells the compiler:

"I intend to override a method from the parent class."

Java will not complain about `sounds()`. It simply treats it as a new method, not an overridden method.

<code>@Override</code>	Result
-----	-----
Present	Method overriding + compiler checks
Not present	Method can still override
Main purpose	Prevent accidental overriding mistakes

Java Interface

An interface in Java is a blueprint that defines a set of methods a class must implement without providing full implementation details. It helps achieve abstraction by focusing on what a class should do rather than how it does it. Interfaces also support multiple inheritance in Java.

- A class must implement all abstract methods of an interface.
- All variables in an interface are public, static, and final by default.
- Interfaces can have default, static, and private methods (Java 8+ and 9+).

Another way to achieve abstraction in Java, is with interfaces.

```
// interface
```

```
interface Animal {
```

```
    public void animalSound(); // interface method  
    (does not have a body)
```

```
    public void run(); // interface method (does not  
    have a body)
```

```
}
```

To access the interface methods, the interface must be "implemented" (kinda like inherited) by another class with the implements keyword (instead of extends).

Java Interface

Notes on Interfaces:

Like abstract classes, interfaces cannot be used to create objects (in the example above, it is not possible to create an "Animal" object in the MyMainClass)

Interface methods do not have a body - the body is provided by the "implement" class

On implementation of an interface, you must override all of its methods

Interface methods are by default abstract and public

Interface attributes are by default public, static and final

An interface cannot contain a constructor (as it cannot be used to create objects)

Why And When To Use Interfaces?

- 1) To achieve security - hide certain details and only show the important details of an object (interface).
- 2) Java does not support "multiple inheritance" (a class can only inherit from one superclass). However, it can be achieved with interfaces, because the class can implement multiple interfaces. Note: To implement multiple interfaces, separate them with a comma (see example below).

Difference Between Class and Interface in Java

Feature	Class	Interface
Purpose	Defines objects, data, and behavior	Defines a contract/rules that a class must follow
Keyword	class	interface
Object creation	Can create objects	Cannot directly create objects
Inheritance	A class can extend one class	A class can implement multiple interfaces
Keyword for inheritance	extends	implements
Variables	Can have instance, static, and final variables	Fields are implicitly public static final
Methods	Can have concrete and abstract methods	Can have abstract, default, static, and private methods
Constructor	Can have constructors	Cannot have constructors
Access modifiers	Methods/fields can use different access modifiers	Interface abstract methods are public by default
Multiple inheritance	Not possible through classes	Possible through multiple interfaces
State	Can maintain object state	Normally defines constants, not object state
Object relationship	Represents what an object is	Represents what an object can do

Difference Between Encapsulation and Abstraction in Java

Feature	Encapsulation(information Hiding)	Abstraction(Hides Implementation)
Meaning	Wrapping data and methods into a single unit	Hiding implementation details and showing only essential features
Main focus	Data security/protection	Complexity hiding
Answers	"How do I protect the data?"	"What should the user see?"
Achieved using	Classes, objects, private fields, getters/setters	Abstract classes and interfaces
Data hiding	Yes	Not its primary purpose
Implementation hiding	Not necessarily	Yes
Access control	Uses private, protected, public	Uses abstract methods/interfaces
Example	private balance with deposit()	Payment.pay() without exposing payment implementation
Real-world example	ATM PIN is protected	ATM shows options without showing internal banking process

Java Abstract Classes

Data abstraction is the process of hiding certain details and showing only essential information to the user.

Abstraction can be achieved with either abstract classes or interfaces.

The abstract keyword is a non-access modifier, used for classes and methods:

- Abstract class: is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).
- Abstract method: can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

Abstract class

An abstract class is a class declared using the abstract keyword that cannot be instantiated directly.

It can contain both abstract methods (without a body) and concrete methods (with a body), allowing it to provide common functionality while leaving some behavior to subclasses.

- Can contain abstract and concrete methods.
- Can have constructors, instance variables, and static members.

Java Abstract Classes

Abstraction in Java is the process of hiding internal implementation details and showing only essential functionality to the user. It focuses on what an object does rather than how it does it.

- It hides the complex details and shows only essential features.
- Abstract classes may have methods without implementation and must be implemented by subclasses.
- By abstracting functionality, changes in the implementation do not affect the code that depends on the abstraction.

Ways to Achieve Abstraction: Java provides two ways to implement abstraction

- Abstract Classes (Partial Abstraction)
- Interface (provides abstraction for behavior, may contain default or static methods)

Java Abstract Classes

Advantages of Abstraction

- Abstraction makes complex systems easier to understand by hiding the implementation details.
- Abstraction keeps different part of the system separated.
- Abstraction maintains code more efficiently.
- Abstraction increases the security by only showing the necessary details to the user.

Limitations of Abstraction in Java

- Can make the code more complex if abstraction is used unnecessarily.
- Can make debugging harder because the implementation details are hidden.
- May require additional classes or interfaces, increasing the amount of code.
- Too many abstraction layers can make the program difficult to understand.
- Designing a good abstraction requires careful planning.
- May add little benefit in small or simple programs.

Access Modifiers in Java

Comparison Table of Access Modifiers in Java

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes